

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Sully Valeria Pihuave Salazar

PARALELO:A

SEMESTRE:Quinto

ACTIVIDAD: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)



PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1

1. Análisis del problema

Actores principales

Actor	Descripción
Usuario final	Personal del Data Center que reporta fallos (hardware, red, software)
Técnico de soporte	Especialista asignado a resolver incidentes (puede haber varios: red, hardware, software)
Administrador de sistemas	Gestiona técnicos, categorías, visualiza métricas y asegura el funcionamiento del sistema

Requerimientos funcionales

- RF-01: Crear ticket con descripción, categoría y prioridad.
- RF-02: Categorizar automáticamente el fallo (Hardware, Red, Software).
- RF-03: Asignar técnico especializado según categoría.
- RF-04: Cambiar estado del ticket (Abierto, En Progreso, Resuelto, Cerrado).
- RF-05: Notificar al técnico cuando se asigna un nuevo ticket.
- RF-06: Listar tickets por técnico, estado o categoría.
- RF-07: Generar reporte de incidentes resueltos.

Requerimientos no funcionales

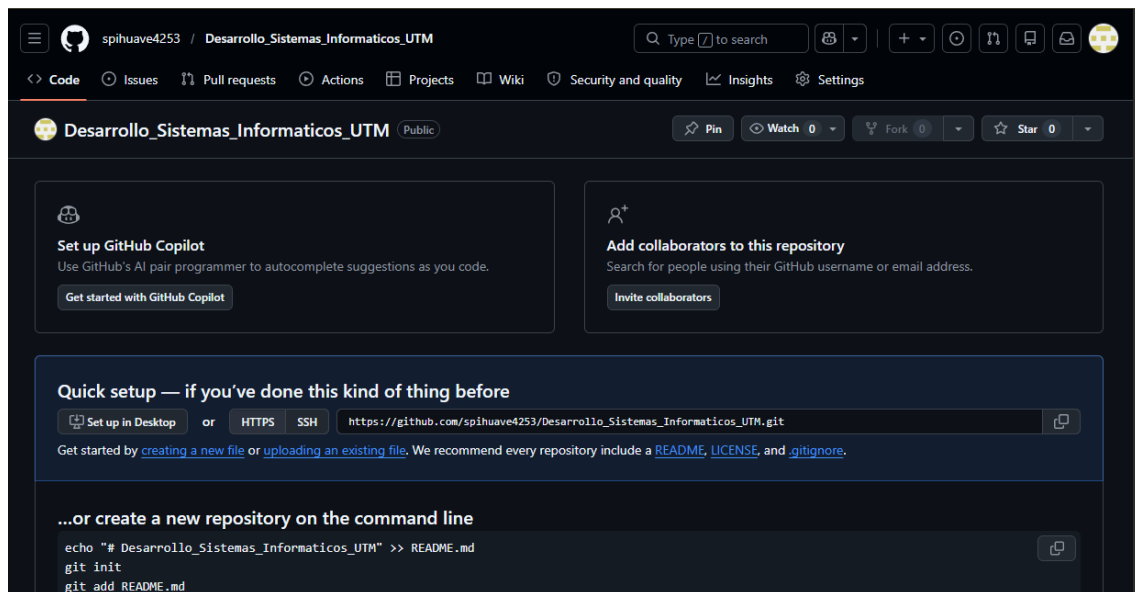
- Escalabilidad: Soporte para cientos de tickets diarios.
- Mantenibilidad: Código organizado con patrones y principios SOLID.
- Trazabilidad de versiones: Uso estricto de Gitflow con ramas feature, develop y main.



2. Diseño del sistema

Creamos nuestro repositorio en GitHub

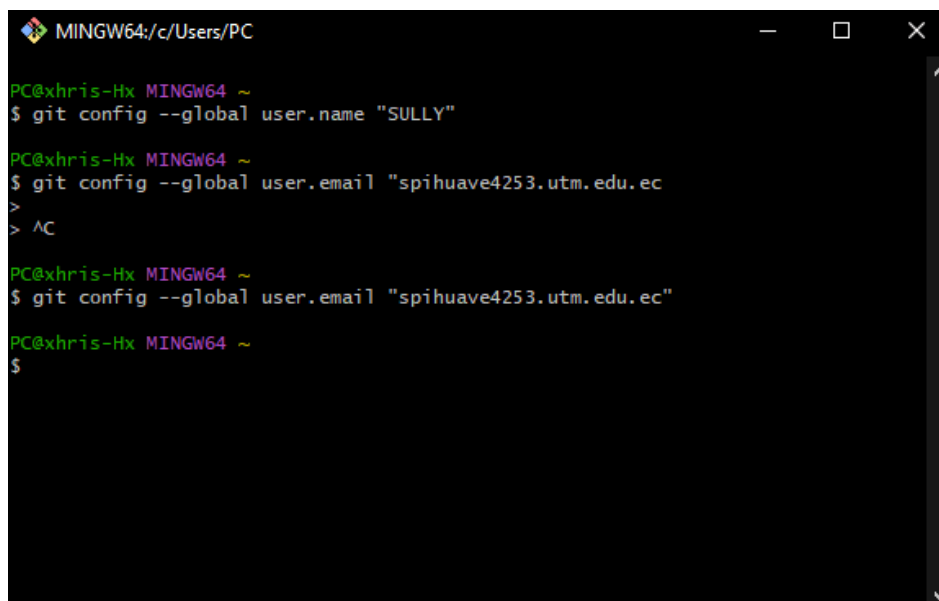
Como primer paso creamos nuestro repositorio en la plataforma de GitHub



Configuracion de la identidad local

```
git config --global user.name "SULLY"
```

```
git config --global user.email "spihuave4253.utm.edu.ec"
```





Inicializamos nuestro init en nuestro repositorio

```
MINGW64:/c/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI
$ git init
Initialized empty Git repository in C:/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI/.git/
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ |
```

Ahora vamos a subir nuestro primer archivo con los comandos

git add.

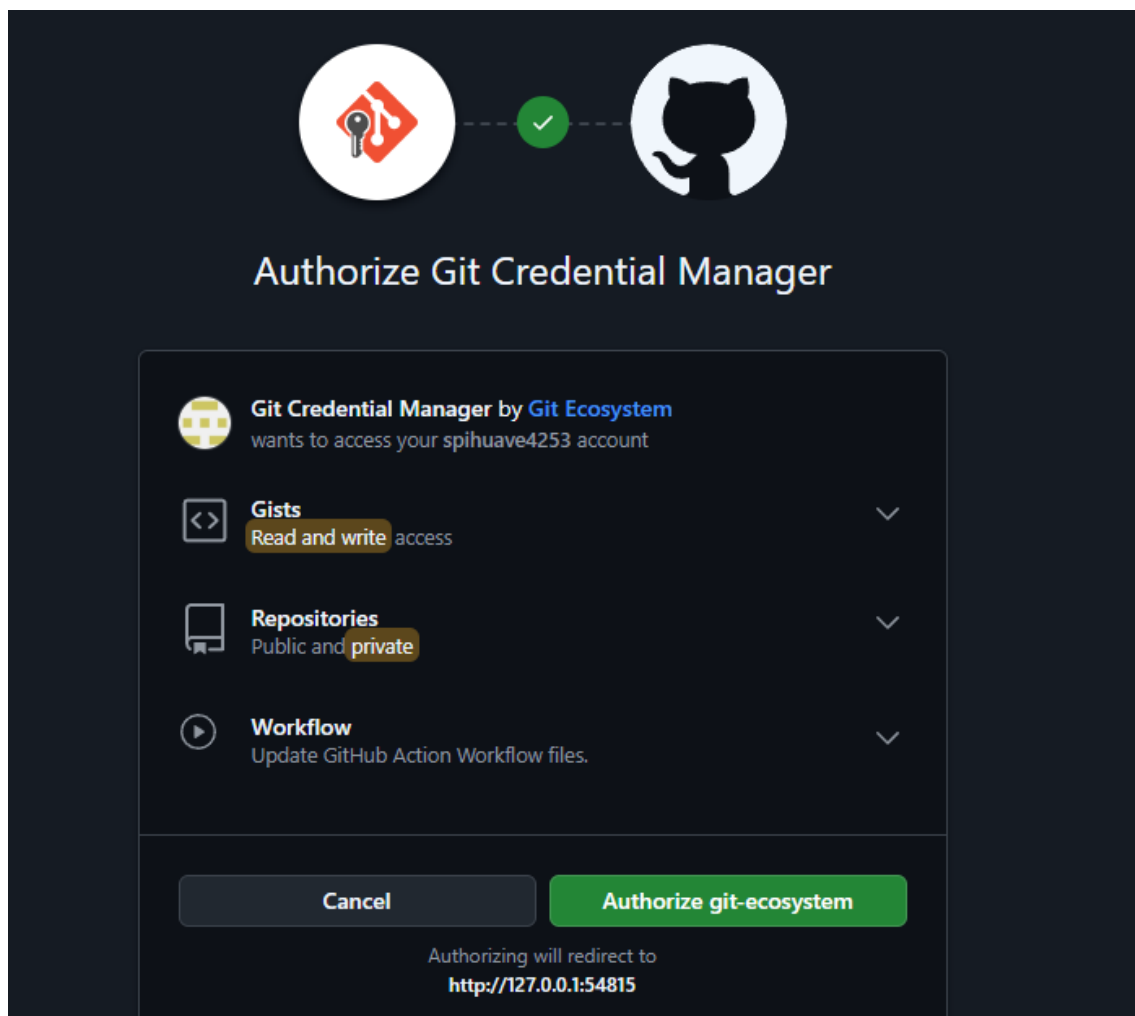
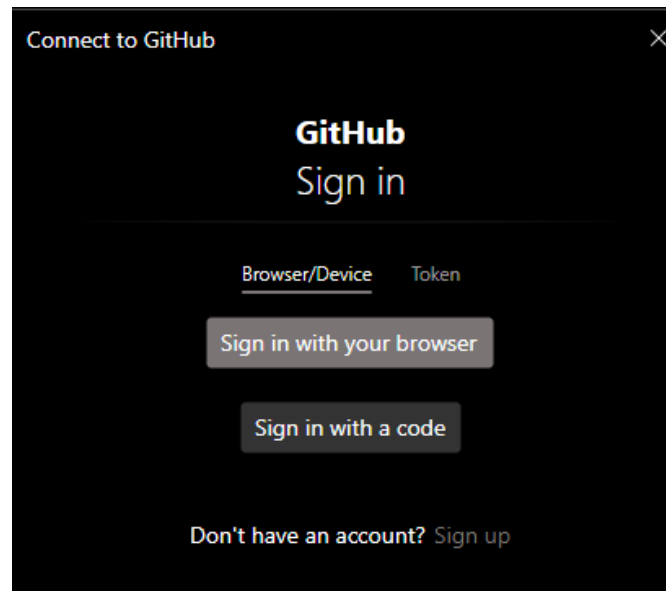
git commit -m "commit inicial: subida de la primera version del pdf"

```
MINGW64:/c/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI
$ git init
Initialized empty Git repository in C:/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI/.git/
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ git add .
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ git commit -m "commit inicial: subida de la primera version del pdf"
[master (root-commit) 6bbc6ad] commit inicial: subida de la primera version del pdf
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Compendio DSI Unidad 1.pdf
PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ |
```

Ahora copiamos la dirección de nuestro repositorio en github para añadirla

https://github.com/spihuave4253/Desarrollo_Sistemas_Informaticos_UTM.git

una vez ejecutada nos redireccionara para iniciar sesión y quedar guardado.





Una vez que confirmemos el inicio de sesión nos saldrá ya la autenticación completa

```
MINGW64:/c/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI
$ AC

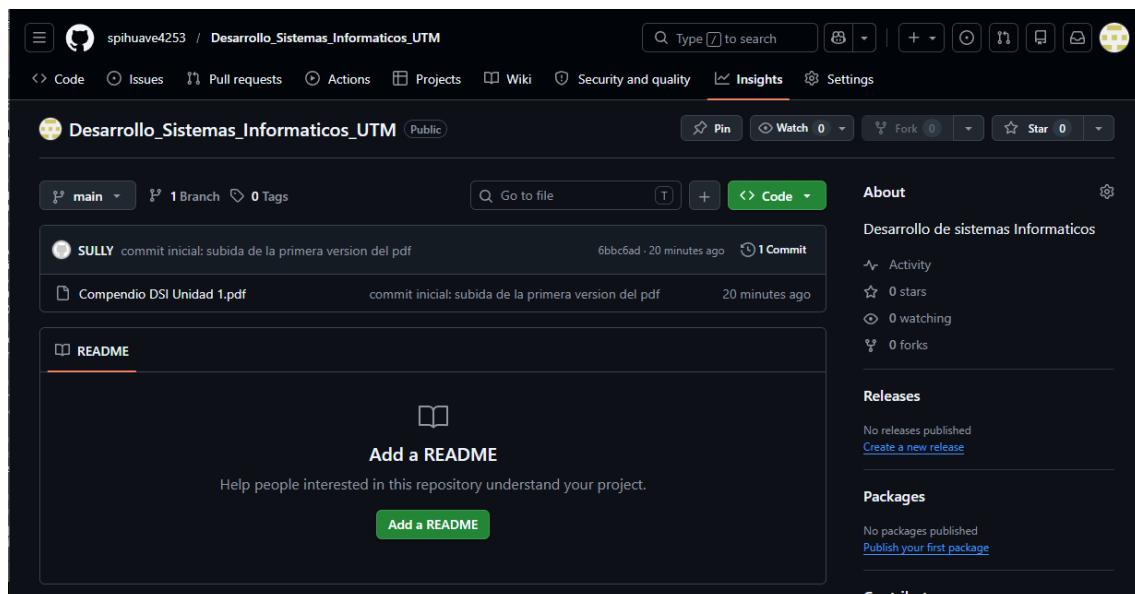
PC@xhrris-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ git remote add origen https://github.com/spihuave4253/Desarrollo_Sistemas_Informaticos_UTM.git

PC@xhrris-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (master)
$ git branch -M main

PC@xhrris-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$ git push -u origen main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 815.25 KiB | 7.41 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/spihuave4253/Desarrollo_Sistemas_Informaticos_UTM.git
 * [new branch]      main -> main
branch 'main' set up to track 'origen/main'.

PC@xhrris-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$
```

Con eso ya ejecutado podemos revisar ya en GitHub si el archivo ya aparece subido



Eso sería todo para poder crear el repositorio, conectarlo y subir los archivos mediante el bash.



Diagrama de casos de uso

Diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.

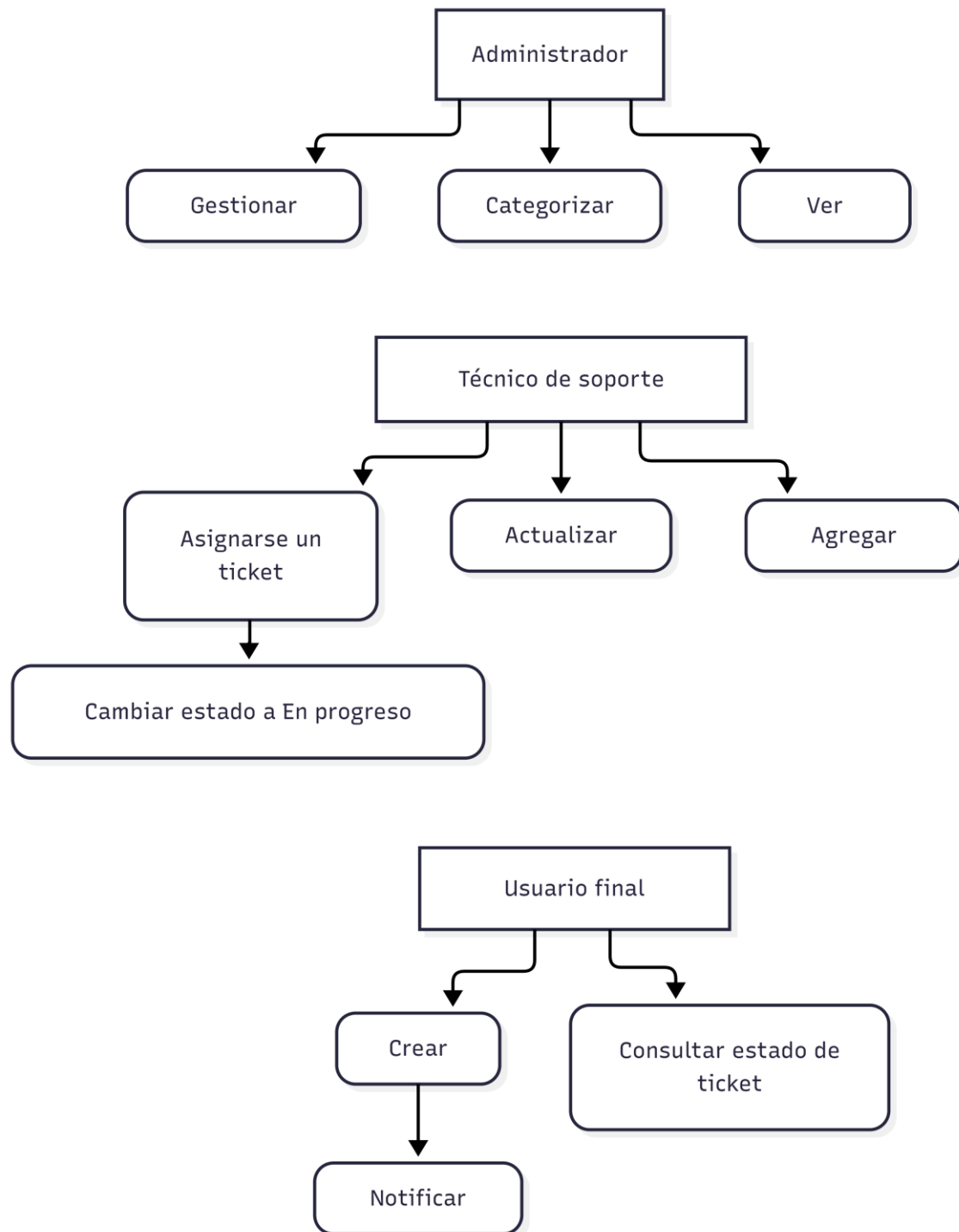
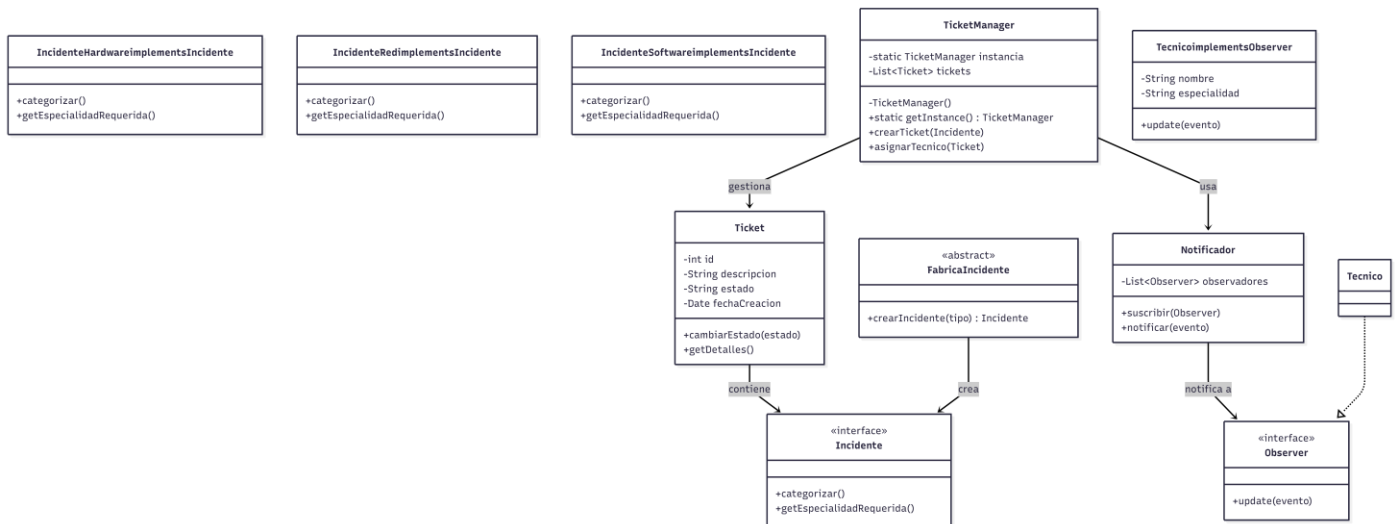




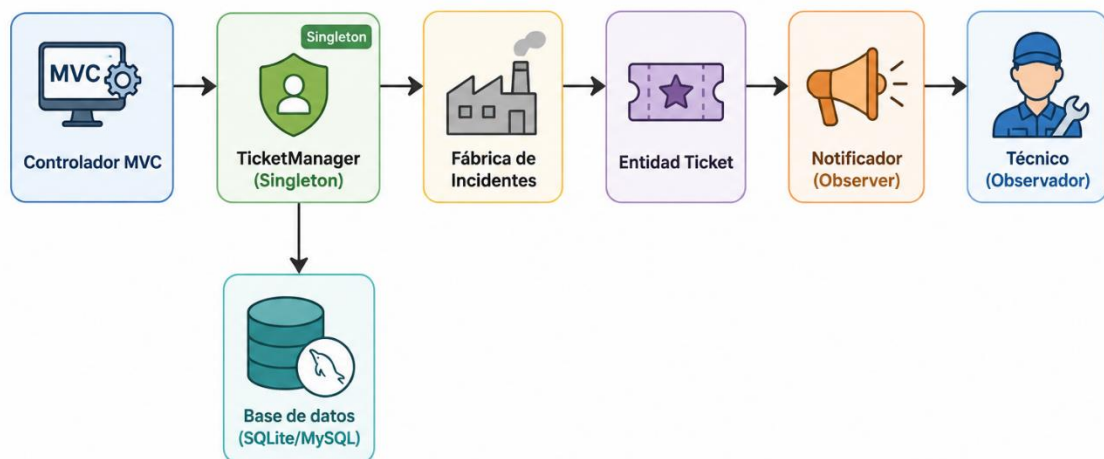
Diagrama de clases UML (con principios SOLID)



Principios SOLID aplicados:

- **SRP:** TicketManager solo gestiona tickets, Notificador solo notifica.
- **OCP:** Podemos agregar nuevos tipos de incidentes sin modificar TicketManager.
- **LSP:** Todas las implementaciones de Incidente son sustituibles.
- **ISP:** Observer tiene un solo método.
- **DIP:** TicketManager depende de Incidente (abstracción), no de clases concretas.

Arquitectura general del sistema





3. Documentación del diseño

Aplicación de patrones de diseño

Patrón 1: Singleton – Motor central de gestión de tickets

- **Propósito:** Garantizar una única instancia del gestor de tickets en todo el sistema, evitando conflictos de concurrencia y consumo excesivo de memoria.
- **Cómo actúa:** En un Help Desk, múltiples usuarios podrían intentar crear tickets simultáneamente, sin Singleton, se crearían múltiples instancias del gestor, causando pérdida de datos, ya que Singleton centraliza el estado.

Patrón 2: Observer – Notificaciones automáticas a técnicos

- **Propósito:** Desacoplar la creación de tickets de la notificación a los técnicos, cuando se crea un ticket, todos los técnicos suscritos reciben la alerta automáticamente.
- **Cómo actúa:** El sistema de Help Desk necesita notificar en tiempo real a los técnicos según su especialidad, con Observer, añadir nuevos tipos de notificaciones (email, SMS, WebSocket) no modifica la lógica central.

Patrón 3: Factory Method – Creación de incidentes por área

- **Propósito:** Encapsular la lógica de creación de los diferentes tipos de incidentes (Hardware, Red, Software) sin que el cliente (TicketManager) conozca las clases concretas.
- **Cómo resuelve:** El Data Center tiene fallos muy distintos. Si usáramos `new IncidenteHardware()` esparcido por el código, cada nuevo tipo (ej. "Fallo eléctrico") obligaría a modificar muchas clases. Factory Method centraliza la creación y cumple OCP.

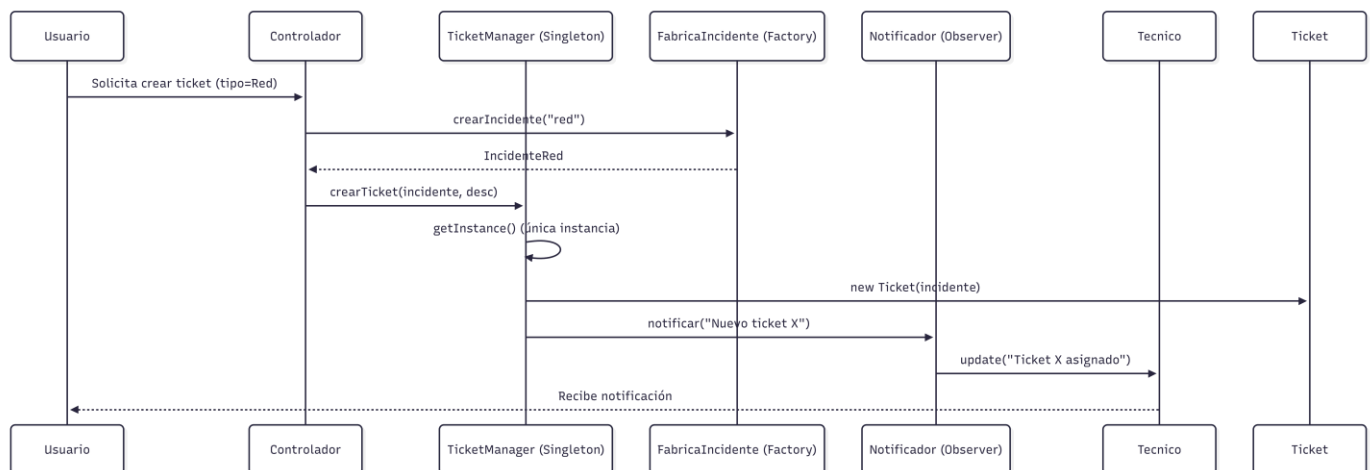


Justificación de Gitflow

El uso de Gitflow destaca para su uso por las siguientes razones:

- Ramas aisladas: feature/ permite desarrollar patrones sin afectar develop.
- Control de versiones: Las etiquetas en main garantizan trazabilidad (ej. v1.0.0 entrega parcial).
- Trabajo en equipo: Simula un entorno real donde varios desarrolladores trabajan en diferentes patrones al mismo tiempo.
- Cumple requisito no funcional: Trazabilidad y mantenibilidad.

Diagramas de clases y secuencia de la implementación de los patrones



Propósito de cada patrón

Patrón	Propósito en Help Desk	Problema resuelto
Singleton	Única instancia del gestor de tickets y notificador	Evita duplicación de tickets y conflictos de concurrencia
Observer	Notificación automática a técnicos cuando se crea/actualiza un ticket	Desacopla la lógica de creación de la de alerta; fácil añadir SMS, email
Factory Method	Creación polimórfica de incidentes según categoría (Hardware/Red/Software)	Permite agregar nuevos tipos de fallos sin modificar el núcleo del sistema



Subida de los diagramas al GitHub

Por último para dar por concluido el trabajo solo queda por subir los diagramas a nuestro repositorio, para eso lo iniciamos en la carpeta con nuestros archivos y utilizamos los siguientes códigos: `git add.` / `git commit -m "Subida de los diagramas (Casos de Uso y UML)"`

```
MINGW64:/c/Users/PC/Documents/Tareas Sully/S5/DSI/repositorioDSI
SI/repositorioDSI/.git/

PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$ git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
    add

PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$ git add .

PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$ git commit -m "Subida de los diagramas (Casos de Uso y UML)"
[main fe64f2d] Subida de los diagramas (Casos de Uso y UML)
4 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 Compendio DSI Unidad 1.pdf
create mode 100644 Diagrama de casos de uso.png
create mode 100644 Diagrama de clases UML (con principios SOLID).png
create mode 100644 "Diagramas de clases y secuencia de la implementaci\303\263n de los patrones.png"

PC@xhri-Hx MINGW64 ~/Documents/Tareas Sully/S5/DSI/repositorioDSI (main)
$
```

